



GUÍA DE APRENDIZAJE

Consumo de APIs con AXIOS en aplicaciones React (Vite)

Tema	AXIOS: importancia, instalación y consumo de APIs con React + Vite
Resultado de aprendizaje	El aprendiz identifica la utilidad de una librería HTTP externa y consume datos de una API pública en una aplicación React construida con Vite, usando AXIOS.
Duración sugerida	5 horas (una jornada de formación)
Modalidad	Presencial

Actividad 1: Reflexión inicial

Duración estimada: 30 minutos

Antes de entrar en materia técnica, tómate unos minutos para reflexionar y responder de forma individual las siguientes preguntas en tu bitácora o cuaderno de apuntes:

1. Cuando usas una aplicación móvil (por ejemplo, una app del clima o de noticias), ¿de dónde crees que provienen los datos que ves en la pantalla si no están escritos directamente en el código?
2. ¿Qué problemas imaginas que enfrentaría un desarrollador si tuviera que "pedir" esa información a otro servidor sin ninguna herramienta que le ayude a organizar esa comunicación?
3. ¿Alguna vez has usado la función fetch de JavaScript? Si es así, ¿qué dificultades recuerdas haber tenido (manejo de errores, cabeceras, conversión de datos, etc.)?

Propósito de esta reflexión

Estas preguntas buscan que el aprendiz relacione su experiencia cotidiana como usuario de aplicaciones con el trabajo técnico que hay detrás: la comunicación entre un cliente (frontend) y un servidor (API). Esta reflexión se socializa brevemente con el grupo antes de avanzar.



Actividad 2: Contextualización e identificación de conocimientos necesarios

Duración estimada: 30 minutos

¿Qué es AXIOS?

Axios es una librería de JavaScript basada en promesas (Promises) que permite realizar peticiones HTTP desde el navegador o desde Node.js. Se utiliza para comunicar una aplicación frontend (como una construida en React) con un servidor o una API, ya sea para obtener información (GET), enviar datos (POST), actualizarlos (PUT/PATCH) o eliminarlos (DELETE).

¿Cuál es su razón de ser?

JavaScript ya cuenta con una función nativa llamada fetch para hacer peticiones HTTP. Sin embargo, Axios surge para resolver varias limitaciones y hacer el trabajo del desarrollador más simple:

- Convierte automáticamente las respuestas a formato JSON, sin necesidad de llamar a .json() manualmente.
- Permite configurar tiempos de espera (timeout) para las peticiones.
- Ofrece un manejo de errores más claro: cualquier código de estado HTTP fuera del rango 2xx entra directamente al bloque de error (catch).
- Permite crear instancias reutilizables con una URL base y cabeceras predefinidas (por ejemplo, para enviar tokens de autenticación en todas las peticiones).
- Soporta interceptores, que permiten ejecutar lógica antes de enviar una petición o después de recibir una respuesta.
- Funciona de forma consistente tanto en el navegador como en entornos de Node.js.

Importancia en el desarrollo profesional

En el mundo laboral, la mayoría de las aplicaciones frontend (React, Vue, Angular) no almacenan la información directamente en el código: la obtienen de un servidor a través de una API REST. Axios es una de las librerías más utilizadas en la industria para esta tarea, por lo que dominarla es una competencia clave para cualquier desarrollador frontend o fullstack.

Conocimientos previos necesarios

Para desarrollar esta guía, el aprendiz debe manejar los siguientes conceptos, revisados en sesiones anteriores:

- Fundamentos de JavaScript: funciones, promesas y async/await.
- Creación de un proyecto React con Vite.
- Componentes funcionales y hooks básicos: useState y useEffect.
- Concepto de API REST y formato JSON.



Actividad 3: Apropiación del conocimiento

Duración estimada: 90 minutos

3.1 Instalación de AXIOS

Axios se instala como una dependencia dentro de un proyecto Node.js/React ya existente. Si aún no tienes un proyecto, primero créalo con Vite:

```
# 1. Crear el proyecto React con Vite
npm create vite@latest mi-app-axios -- --template react

# 2. Entrar a la carpeta del proyecto
cd mi-app-axios

# 3. Instalar las dependencias del proyecto
npm install

# 4. Instalar Axios
npm install axios

# 5. Levantar el servidor de desarrollo
npm run dev
```

Una vez instalado, Axios queda disponible como cualquier otro paquete de node_modules y se importa donde se necesite con:

```
import axios from 'axios';
```

3.2 Ejemplo básico: consumo de una API con Axios en React + Vite

A continuación se presenta el ejemplo guía visto en clase: un componente que consume una API pública de usuarios de prueba (JSONPlaceholder) y muestra la lista en pantalla, controlando los estados de carga y error.

Archivo: `src/App.jsx`

```
import { useState, useEffect } from 'react';
import axios from 'axios';

function App() {
  const [usuarios, setUsuarios] = useState([]);
  const [cargando, setCargando] = useState(true);
```



```
const [error, setError] = useState(null);

useEffect(() => {
  axios
    .get('https://jsonplaceholder.typicode.com/users')
    .then((respuesta) => {
      setUsuarios(respuesta.data);
      setCargando(false);
    })
    .catch((error) => {
      setError('No se pudo cargar la información de la API');
      setCargando(false);
    });
}, []);

if (cargando) return <p>Cargando usuarios...</p>;
if (error) return <p>{error}</p>;

return (
  <div>
    <h1>Lista de usuarios</h1>
    <ul>
      {usuarios.map((usuario) => (
        <li key={usuario.id}>
          {usuario.name} – {usuario.email}
        </li>
      ))}
    </ul>
  </div>
);
}

export default App;
```

Explicación del ejemplo:

- `useEffect` con arreglo de dependencias vacío (`[]`): ejecuta la petición una sola vez, cuando el componente se monta.
- `axios.get(url)`: realiza la petición GET a la API y devuelve una promesa.
- `respuesta.data`: Axios ya entrega el cuerpo de la respuesta convertido a objeto/arreglo JavaScript, sin necesidad de parsear JSON manualmente.



- Estados cargando y error: permiten mostrarle al usuario retroalimentación mientras se resuelve la petición o si esta falla.
- `usuarios.map(...)`: recorre el arreglo de datos recibido y genera dinámicamente la lista en el DOM.

3.3 Ejercicio propuesto.

Con el apoyo del instructor para dudas , y siguiendo el ejemplo anterior, realiza los siguientes pasos:

4. Crea un nuevo proyecto con Vite y react, e instala Axios siguiendo el procedimiento de la sección 3.1.
5. Copia el componente `App.jsx` del ejemplo básico y verifica que la lista de usuarios se muestre correctamente en el navegador.
6. Cambia la URL de la petición por `https://jsonplaceholder.typicode.com/posts` y ajusta el componente para mostrar el título (title) de cada publicación.
7. Agrega un `console.log(respuesta)` dentro del `.then()` para observar en las herramientas de desarrollador la estructura completa del objeto que retorna Axios (data, status, headers, config).

Actividad 4: Transferencia del conocimiento

Duración estimada: 45 minutos

De forma individual o en parejas, el aprendiz debe crear un nuevo proyecto React con Vite y desarrollar un componente que consuma una API pública distinta a la usada en el ejemplo guía, aplicando lo aprendido sobre Axios.

Requisitos del producto a entregar

- Proyecto creado con Vite (`npm create vite@latest`) y Axios instalado como dependencia.
- Uso de `useEffect` y `useState` para gestionar la petición y el estado del componente.
- Manejo de al menos tres estados de la interfaz: cargando, error y datos cargados correctamente.
- Renderizado dinámico de la información obtenida usando `.map()`.
- Código comentado explicando, con tus propias palabras, qué hace cada bloque relevante (especialmente la llamada a Axios).



Sugerencias de APIs públicas gratuitas para usar

- <https://pokeapi.co/api/v2/pokemon> (información de Pokémon)
- <https://rickandmortyapi.com/api/character> (personajes de una serie)
- <https://api.thecatapi.com/v1/images/search> (imágenes de gatos)
- <https://jsonplaceholder.typicode.com/todos> (lista de tareas de prueba)

NOTA : Puede usar las APIs de la sección anterior o consultar otras relacionadas ojala con su proyecto formativo.

Evidencia de aprendizaje

El aprendiz debe entregar el enlace al repositorio en GitHub con el proyecto completo (incluyendo package.json y el código fuente, sin la carpeta node_modules) y estar en capacidad de explicar o sustentar oralmente, ante el instructor, el funcionamiento del componente desarrollado.

Criterios de evaluación

- Instalación correcta de Axios y configuración del proyecto con Vite.
- Correcto uso de axios.get() y manejo de la promesa devuelta (then/catch o async/await).
- Manejo adecuado de los estados de carga y error, evidenciando comprensión del ciclo de vida de una petición HTTP.
- Claridad y organización del código, con comentarios pertinentes.
- Capacidad de explicar el funcionamiento del ejercicio ante el instructor.